

The Ostrich Algorithm

1 Main Idea

Definition

The **Ostrich algorithm** is a deadlock-handling strategy in which the operating system simply ignores the deadlock problem.

The name is based on the idea of hiding one's head in the sand and pretending the problem does not exist. In operating systems, this means no special deadlock detection, avoidance, or prevention is performed.

Core Point

The Ostrich algorithm is not a real algorithm for solving deadlocks. It is a policy choice: ignore rare deadlocks because solving them may cost more than tolerating them.

2 Why Ignore Deadlocks?

Deadlock handling can be expensive. Detection needs runtime checks, avoidance needs information about future resource needs, and prevention can reduce flexibility or performance.

Reason	Explanation
Deadlocks may be rare	If deadlocks happen very infrequently, the cost of preventing every one may not be justified.
Handling has overhead	Detection and avoidance algorithms consume CPU time, memory, and bookkeeping effort.
Prevention restricts programs	Forcing strict rules, such as fixed resource ordering, can make systems less convenient or less efficient.
Recovery may be simple	A user or administrator can kill and restart a stuck program when the problem occurs.

3 When It Makes Sense

Practical Use

The Ostrich algorithm is reasonable when deadlocks are rare, the damage is small, and the cost of full deadlock handling is high.

Many general-purpose operating systems use this practical attitude for some kinds of deadlocks. They do not try to prevent every possible deadlock in user programs because doing so would make the system more complex and slower for normal cases.

Example

If two user programs occasionally deadlock while using files or locks, the system may simply let the user terminate one of them. This is often cheaper than constantly tracking every possible deadlock.

4 When It Is Not Acceptable

Do not use blindly

Ignoring deadlocks is dangerous when failure is frequent, costly, or safety-critical.

System type	Why ignoring is bad
Real-time control system	A deadlock may miss a deadline and cause physical damage.
Banking or transaction system	A deadlock can block important operations and leave users waiting.
Database server	Deadlocks may be common enough that detection and rollback are necessary.
Medical or aviation system	Manual restart is not an acceptable recovery plan.

5 Comparison with Other Strategies

Strategy	Main idea	Cost
Ostrich algorithm	Ignore the problem.	Lowest overhead, but no protection.
Detection and recovery	Let deadlocks occur, detect them, then recover.	Runtime checks and recovery work.
Avoidance	Grant a request only if the resulting state is safe.	Needs advance information and decision logic.
Prevention	Break at least one Coffman condition structurally.	Can restrict resource use and reduce flexibility.

6 Simple Decision Rule

Use or Avoid

Use Ostrich	if deadlocks are rare and cheap to recover from.
Do not use Ostrich	if deadlocks are frequent, costly, or safety-critical.

7 Worked Example

Problem

A desktop operating system has a rare deadlock in a user application once every few months. Preventing it would require expensive resource tracking for every program. Should the system use the Ostrich algorithm?

Solution

The deadlock is rare and affects only a user application. The user can terminate and restart the application. Constant deadlock prevention would add overhead to every normal execution.

Answer

Using the Ostrich algorithm is reasonable here because the cost of full prevention is greater than the cost of occasional manual recovery.

8 Common Mistakes

Mistake 1

Do not say the Ostrich algorithm prevents deadlocks. It does not prevent, detect, or recover from them automatically.

Mistake 2

Do not assume ignoring deadlocks is always careless. It can be a rational engineering choice when deadlocks are extremely rare.

Mistake 3

Do not use this strategy for systems where a deadlock can cause serious loss, missed deadlines, or unsafe behavior.